

NATIONAL INSTITUTE OF TECHNOLOGY CALICUT



MINOR PROGRAMME ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

WINTER 2026

EE3058E Essentials of AI and Machine Learning

PROJECT

Deep Learning-Based Power Side-Channel Analysis on AES:
A Comparative Study of MLP, CNN and CNN-LSTM on
Masked and Unmasked Implementations

COURSE FACULTY:

Dr. SHIHABUDHEEN K. V.

SUBMITTED BY:

ASWANTH GOPAL — B240523EC

Contents

1	Abstract	3
2	Introduction	3
3	Literature Review	4
4	Problem Statement and Theory	4
4.1	Problem Statement	4
4.2	The Theory of Power Side-Channel Attacks	5
4.3	Why target the S-Box?	5
4.4	The Hamming Weight Leakage Model	5
4.5	System Architecture	5
4.6	Deep Learning Architectures and Implementation	6
4.6.1	Multi-Layer Perceptron (MLP)	6
4.6.2	1D Convolutional Neural Network (1D-CNN)	7
4.6.3	Hybrid CNN-LSTM	7
5	Dataset Description	7
5.1	Dataset Sources and Characteristics	8
5.2	Feature Details and Data Shapes	8
5.3	Class Details (The Target Variable)	8
5.4	Trace Visualization	8
6	Results and Evaluation	9
6.1	Multi-Layer Perceptron (MLP) Results	9
6.2	1D Convolutional Neural Network (1D-CNN) Results	10
6.3	Hybrid CNN-LSTM Results	11
6.4	Summary of Findings	12
7	Conclusion	13
A	Appendix: Implementation Codes	13
A.1	Code A: Dataset Download and Extraction	13
A.2	Code B: Power Trace Visualization	14
A.3	Code C: Multi-Layer Perceptron on ASCAD Dataset	15
A.4	Code D: Multi-Layer Perceptron on AES_HD Dataset	17
A.5	Code E: 1D Convolutional Neural Network on AES_HD Dataset	19
A.6	Code F: Hybrid CNN-LSTM on AES_HD Dataset	21

List of Figures

1	System Architecture Flowchart for Power Side-Channel Attack	6
2	Sample Raw Power Traces	9
3	MLP Guessing Entropy (AES_HD)	10
4	MLP Accuracy (AES_HD)	10
5	CNN Guessing Entropy (AES_HD)	11
6	CNN Accuracy (AES_HD)	11
7	Hybrid CNN-LSTM Guessing Entropy (AES_HD)	12
8	Hybrid CNN-LSTM Accuracy (AES_HD)	12

1 Abstract

This project evaluates three deep learning architectures, MLP, CNN, and hybrid CNN-LSTM, for power side-channel analysis on AES 128. All three models were tested against a noisy hardware (FPGA)-implemented AES-HD dataset, and additionally, MLP was tested on a software-implemented, boolean-masked ASCAD dataset to analyze how well the model works on masked datasets. The result showed that MLP failed completely on the ASCAD dataset, yielding an accuracy of 0.43%, which is equivalent to random guessing. However, on the AES-HD dataset, both the CNN and the MLP successfully recovered the key. The MLP succeeded mainly because the data was carefully scaled. But the CNN-LSTM hybrid was a failure. Even though the CNN hybrid and the CNN both achieved an accuracy of 26.6%, the hybrid failed, indicating that accuracy is a highly misleading metric in this context, while guessing entropy serves as the true measure.

2 Introduction

Cryptographic algorithms like AES 128 (Advanced Encryption Standard) are mathematically unbreakable, but the silicon chips on which they run have a loophole to break them without using the unbreakable math. When a microcontroller or FPGA encrypts data, its transistors leak power. Using these power traces, hackers can reverse engineer it to discover the secret key. Exploiting this vulnerability is known as power side-channel analysis (SCA).

Over the last few years, research has been ongoing to train neural networks to predict this key as well as to determine which points in the power traces are leaking the information. Still there are debates among which model does this well, especially in deliberately included security measures and the noisy hardware.

In this project I evaluated three deep learning architectures: CNN 1D (Convolutional Neural Networks), MLP (Multi-layer Perceptron), and CNN-LSTM hybrid. I tested these models on two different datasets. The first one, ASCAD, which is a protected (boolean masked) dataset, features a software AES implementation (executed on an ATMEGA microcontroller). The second one is AES-HD, which is an unmasked noisy dataset (executed on FPGA).

The objective of this project is to analyze how the above mentioned models work on the above two datasets. Additionally, this project aims to demonstrate that metrics like accuracy are highly misleading in measuring attack success, whereas guessing entropy truly measures the attack success.

3 Literature Review

Kocher et al. [1] started the study of physical vulnerabilities in cryptography. Through the introduction of Differential Power Analysis (DPA), they proved that using statistical observation, mathematical security can be bypassed completely, establishing power side-channel analysis as a crucial threat.

For over a decade attackers relied on manual statistical models. However, the research conducted by Maghrebi et al. [2] established that neural network models like MLPs and CNNs can extract the leakage features.

As deep learning gained popularity, researchers required standardized benchmarks to test different architectures. Prouff et al. [3] published the ASCAD dataset, which features a software-based AES implementation safeguarded by first-order Boolean masking. Zaid et al. [4] shifted focus to unprotected noisy hardware. They worked on optimizing deep learning models for Field Programmable Gate Arrays (FPGAs). They demonstrated that properly sized sliding filters in a 1D CNN could isolate Hamming weight leakages despite massive trace misalignment.

Benadjila et al. [5] conducted a study and concluded that the guessing entropy must be the primary metric for evaluating architecture success rather than accuracy, as an attacker’s aim is to find the key rather than a class prediction.

Integrating these methodologies, I evaluated an MLP against ASCAD’s masking countermeasures and tested CNN, CNN-LSTM, and MLP architectures against the hardware noise of AES.HD. Following Benadjila et al. [5], I utilized guessing entropy rather than accuracy as the metric for attack success.

4 Problem Statement and Theory

4.1 Problem Statement

The aim of this project is to perform a power side-channel attack to recover the secret key of the AES-128 encryption algorithm. AES encrypts data by processing it through a series of complex mathematical operations. Using deep learning models, my goal was to map the physical power consumption of a cryptographic device back to intermediate mathematical values it is calculating. By identifying these intermediate values (S-Box output), the secret key can be mathematically reverse-engineered. For this purpose, I have made use of different neural network architectures (MLP, 1D-CNN, and a CNN-LSTM hybrid) on the AES.HD dataset and MLP on the ASCAD dataset.

4.2 The Theory of Power Side-Channel Attacks

Mathematical encryptions are theoretically unbreakable, but when the encryption runs on hardware during the process, it flips the bits. When the bits flip, it leaks power. This consumption is directly proportional to the Hamming weight (the number of 1's) of the data being processed. By using an oscilloscope, power traces at a particular instance (s-box output) are captured. For thousands of plaintexts, the power traces will be calculated and fed to the ML model. If the power leakage has a direct correlation with the key, the key can be found by reverse engineering, and the encryption can be broken. In real hardware (AES_HD dataset), the power spikes don't always happen at the exact same microsecond, which is why I used CNN (shift-invariance) and highly optimized MLP.

4.3 Why target the S-Box?

The AES-128 key size is 128 bits, and it encrypts the data through 10 rounds of unbreakable mathematical transformation. In the initial round, the plain text is xored with the secret key, and the resulting xored value is passed into the substitution box (S-box), a lookup table that replaces the input with a different output byte. The S-box is the primary target for side channel attacks because it is the only nonlinear step in AES; changing even a single bit of input causes a highly visible flip in the physical hardware registers (Avalanche Effect). Another reason is as it requires both plain text and the unknown key, discovering the S-box output allows the attacker to find the key byte used by reversing the logic.

4.4 The Hamming Weight Leakage Model

This project utilizes the Hamming weight model to connect the power trace to mathematical S-box output. The Hamming weight is the number of 1 bits in a binary sequence. The dynamic power consumed by a chip at the exact moment the S-Box calculates its output is directly proportional to that Hamming weight. Hamming weight can range from 0 to 8, as the AES byte is 8 bits. The problem now changes to a 9-class machine learning classification problem. By targeting 9 Hamming weight classes rather than all 256 possible byte values, the model can more effectively learn the underlying physical leakage patterns despite the presence of hardware noise.

4.5 System Architecture

To automate this attack, the methodology follows the architecture outlined in the flowchart below.

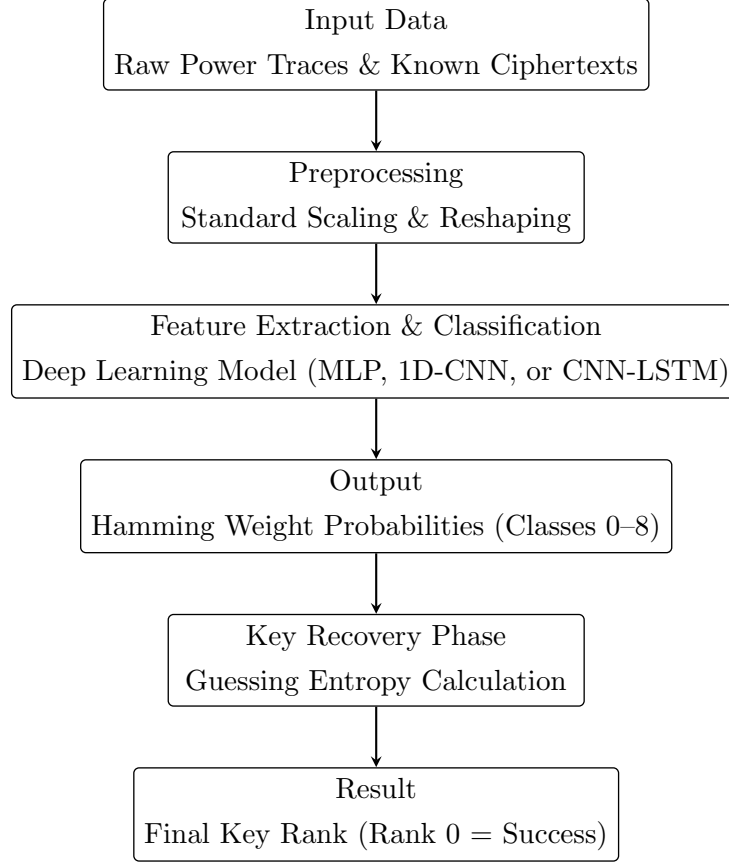


Figure 1: System Architecture Flowchart for Power Side-Channel Attack

4.6 Deep Learning Architectures and Implementation

All the three neural networks were implemented using TensorFlow/Keras. Datasets were preprocessed using standard scaling and optimized using the Adam optimizer with dynamic learning rate reduction. The final output layer of all the models consists of a Dense layer with 9 neurons using the Softmax activation function to get the probability distribution for the 9 possible Hamming weight classes.

4.6.1 Multi-Layer Perceptron (MLP)

Basic Theory: MLP is a feed-forward artificial neural network that consists of multiple layers of nodes (neurons). Every neuron of one layer is fully connected to every other neuron in the next layer. The method of backpropagation is used by this model to learn by adjusting the weights of these connections. It maps complex, non-linear inputs to outputs. In theory, an MLP should fail on noisy hardware because of its fixed structure. I used it as a baseline to see if proper data scaling and tuning could make it succeed.

Architecture and Code Implementation: The input of the MLP is 1D arrays of 1,250 timepoints. The network has the following features: three hidden layers, each contain-

ing 256 neurons with ReLU (Rectified Linear Unit) activation functions, a dropout layer with a rate of 0.2, and batch normalization, which were used to prevent noise overfitting and to ensure stable training.

4.6.2 1D Convolutional Neural Network (1D-CNN)

Basic Theory: CNN works by applying kernels (mathematical filters) across the time-series data. Convolution is performed as this filter sweeps across the input. It extracts specific shapes and learns local spatial hierarchies regardless of where they appear in the trace. This property of shift invariance makes this model robust against clock misalignment found in the raw FPGA datasets.

Architecture and Code Implementation: Here the input for the CNN is a 3D tensor (1250, 1). All the convolutions in my model utilize a kernel size of 11 and ReLU activation. The feature extraction block consists of three sequential Conv1D layers with 8, 16, and 32 filters, respectively. For down sampling the spatial dimensions, each convolutional layer is followed by a max pooling 1D layer and batch normalization. After extracting feature maps, they are passed through two dense layers (64 and 32 neurons) using batch normalization, ReLU, and a dropout of 0.4.

4.6.3 Hybrid CNN-LSTM

Basic Theory: I used this architecture to combine the spatial feature extraction feature of the CNN and the sequential memory property of LSTM. Long Short-Term Memory (LSTM) is a special type of Recurrent Neural Network (RNN). LSTM can remember or forget information over long sequences. They have internal gates, which allow them to do so.

Architecture and Code Implementation: This hybrid model has a CNN feature extractor identical to the first two blocks of the standard CNN (16 and 32 filters with MaxPooling). This compresses the raw trace from 1,250 timepoints down to a dense sequence of 78 clean features. This sequence is fed directly into an LSTM layer containing 64 units. After regularization with batch normalization and a 0.4 dropout, the output of the LSTM is passed to the final 64-neuron dense classification layer.

5 Dataset Description

For this project I have used two standard cryptographic benchmark datasets: **AES_HD** and **ASCAD**.

5.1 Dataset Sources and Characteristics

- **AES_HD (Hardware Dataset):** This dataset represents an unprotected FPGA hardware implementation of AES-128. The power traces were captured using an oscilloscope targeting the encryption process. Because it is raw hardware, it contains severe clock jitter and high-frequency background noise.
- **ASCAD (Software Dataset):** The ASCAD dataset represents a software-based implementation of AES running on an 8-bit ATmega8515 microcontroller. This dataset is protected by a first-order Boolean masking countermeasure, designed to break the correlation between the power consumption and the processed data.

5.2 Feature Details and Data Shapes

The raw datasets were loaded as NumPy arrays (.npy files) and split into independent profiling (training) and attack (testing) sets.

- **AES_HD Structure:** The raw traces contain 1,250 temporal features (time-points) per trace. The training set consists of 50,000 traces, and the test set consists of 25,000 traces.
- **Data Reshaping:** For the MLP, data was input as a 1D vector of shape(1250,). For the CNN and the hybrid model, data was reshaped to (1250, 1) to allow spatial filtering. All the traces were normalized using Standard Scaler to ensure stable gradient convergence.

5.3 Class Details (The Target Variable)

The raw labels in the datasets correspond to the 0-255 value of the intermediate byte value of the S-Box output. These labels were converted into Hamming Weights as the power consumption correlates to the number of active bits.

A custom vectorization function was applied to count the number of '1's in the binary representation of each label. Now the problem changed to a 9-class classification problem. The labels were then One-Hot Encoded to be compatible with the Softmax output layer of the neural networks.

5.4 Trace Visualization

To understand the nature of the physical leakage, a sample of the raw power traces was visualized.

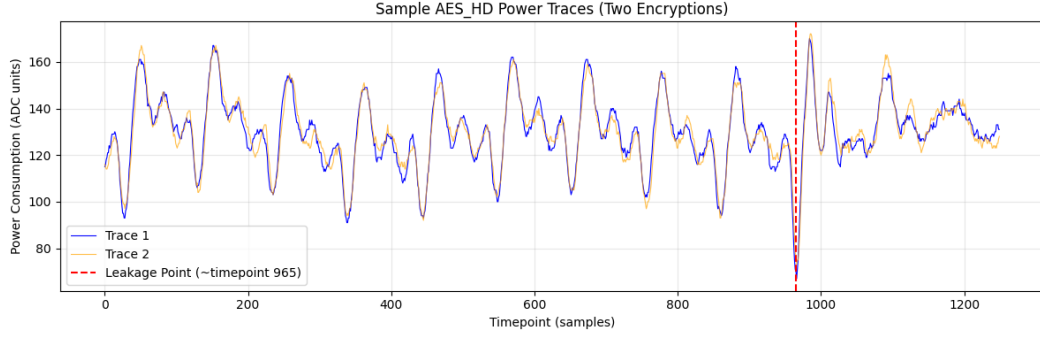


Figure 2: Sample Raw Power Traces

6 Results and Evaluation

Test Accuracy (the model’s ability to classify the Hamming Weight of the S-Box output) and Guessing Entropy (the rank of the true secret key as more traces are processed) were the primary metrics used to evaluate the models. An initial MLP test was also run on the ASCAD dataset to analyze the effects of Boolean masking countermeasures.

6.1 Multi-Layer Perceptron (MLP) Results

The MLP was first tested on the ASCAD dataset, resulting in an accuracy of 0.43% (equivalent to random guessing), which was a complete attack failure. However, it performed remarkably well on the unmasked AES_HD dataset.

- **Classification:** On AES_HD, the MLP achieved a final test accuracy of 26.04%. This represents a highly significant 14.92% improvement over random guessing (11.11% for a 9-class problem).
- **Key Recovery:** Even though clock jitter of hardware possessed a threat to mlp success, standard scaling and batch normalization allowed the network to isolate the leakage. As shown in the Guessing Entropy plot below, the MLP successfully drove the true key to Rank 0 after processing the 25,000 attack traces, successfully cracking the AES key.

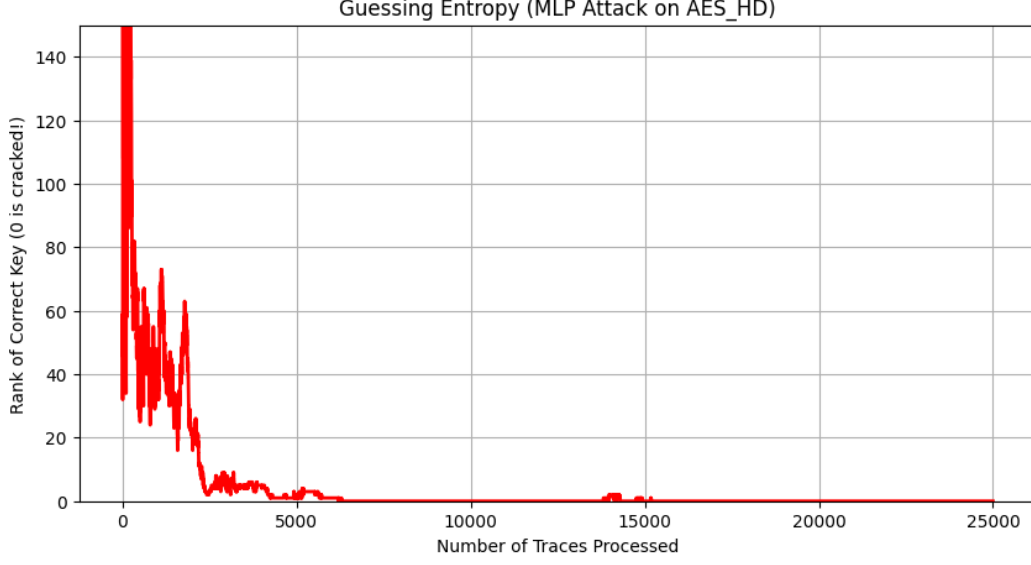


Figure 3: MLP Guessing Entropy (AES_HD)

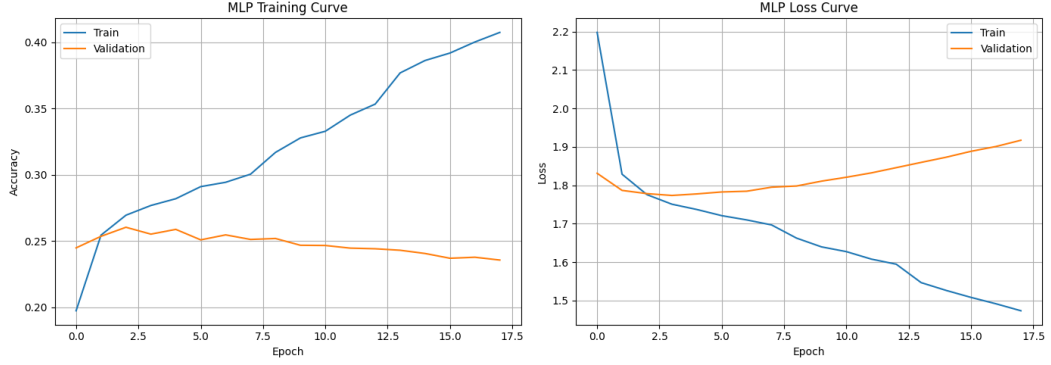


Figure 4: MLP Accuracy (AES_HD)

6.2 1D Convolutional Neural Network (1D-CNN) Results

The 1D-CNN naturally resolved the clock jitter issue in the AES_HD traces by utilizing its shift-invariant mathematical filters.

- **Classification:** The CNN achieved a validation accuracy of approximately 26.70% before triggering early stopping.
- **Key Recovery:** The spatial filtering of the CNN easily bypassed the hardware noise. The Guessing Entropy smoothly converged to the correct key to rank 0.

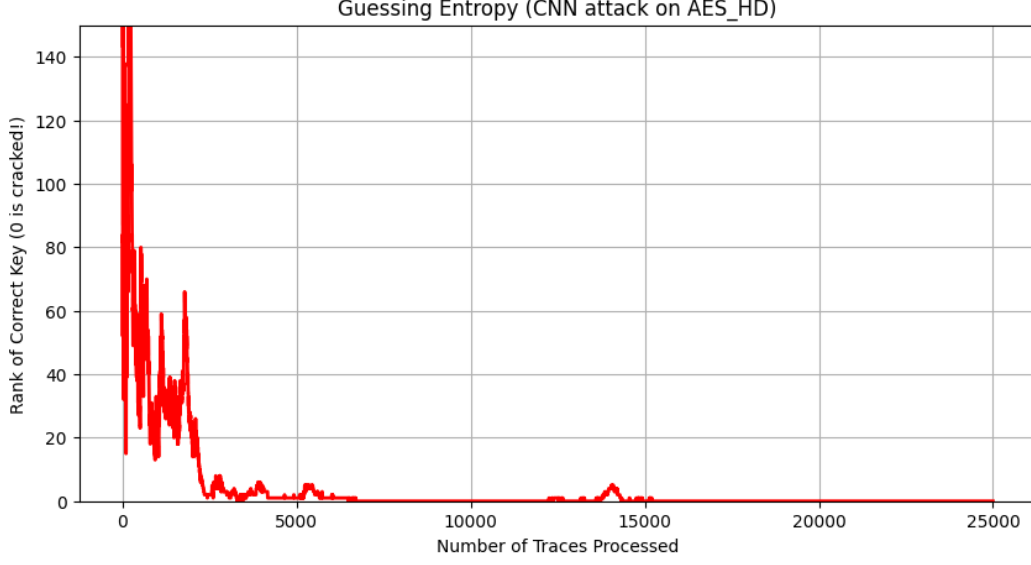


Figure 5: CNN Guessing Entropy (AES_HD)

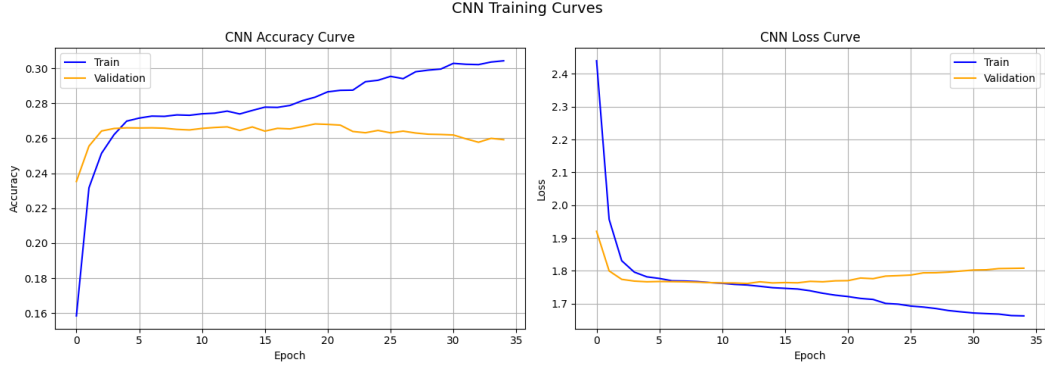


Figure 6: CNN Accuracy (AES_HD)

6.3 Hybrid CNN-LSTM Results

- **Classification:** This model achieved a validation accuracy of approximately 26.65%.
- **Key Recovery:** Despite having strong classification accuracy, the key recovery attack failed, with the true key ending at Rank 68. From this failure we can see that AES S-Box power leakage is an instantaneous physical event rather than a sequential pattern. When CNN-extracted spatial features are fed into an LSTM, it destroys the location-specific data required by the Bayesian key recovery algorithm.

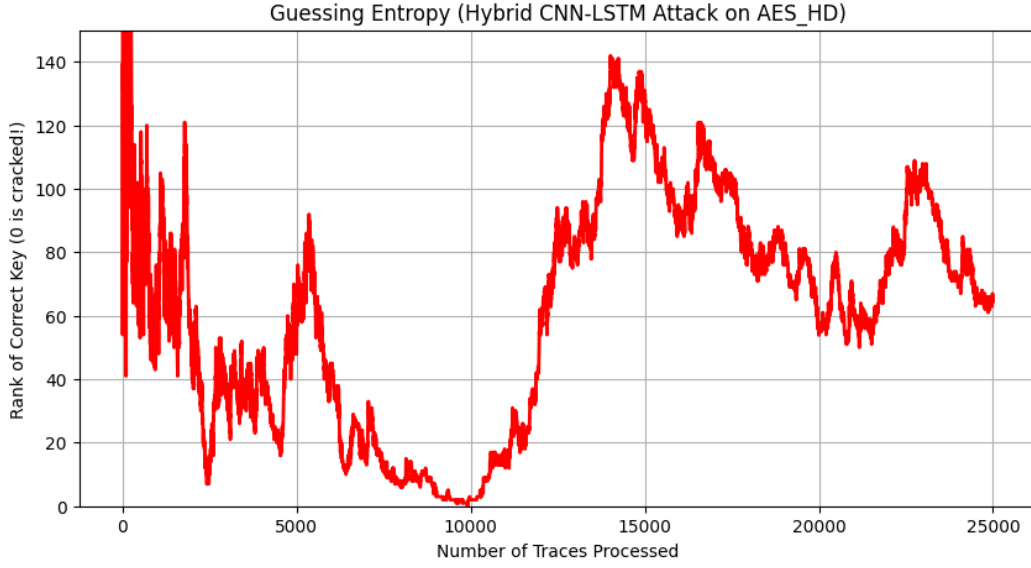


Figure 7: Hybrid CNN-LSTM Guessing Entropy (AES_HD)

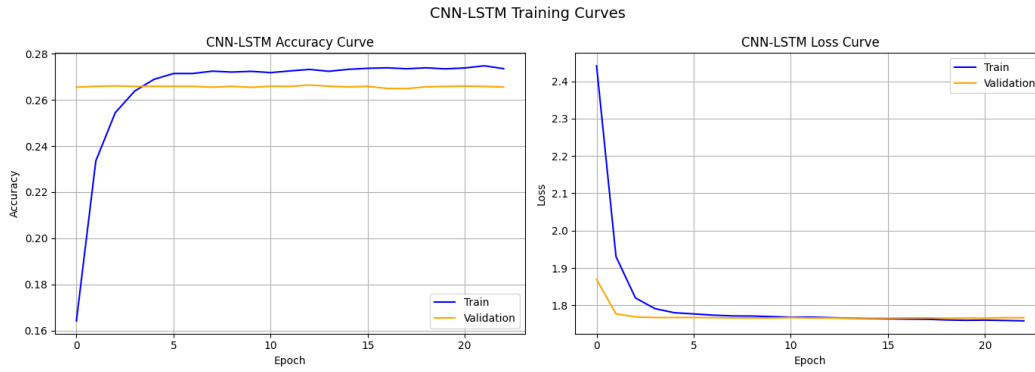


Figure 8: Hybrid CNN-LSTM Accuracy (AES_HD)

6.4 Summary of Findings

The table below summarizes the final performance metrics across all tested architectures on the AES_HD dataset.

Table 1: Performance Comparison on AES_HD Dataset

Model Architecture	Max Validation / Test Accuracy	Final Key Rank	Attack Status
MLP (Baseline)	26.04%	Rank 0	Success
1D-CNN	26.70%	Rank 0	Success
Hybrid CNN-LSTM	26.65%	Rank 68	Failed

7 Conclusion

I evaluated the efficacy of three deep learning architectures—MLP, 1D-CNN, and a Hybrid CNN-LSTM—in performing power side-channel attacks against the AES-128 encryption algorithm.

First, the experiment proved that test accuracy is a highly misleading metric in side-channel analysis. While the CNN and the Hybrid CNN-LSTM achieved nearly identical classification accuracies ($\tilde{26.6\%}$), the CNN successfully cracked the key while the Hybrid model failed completely. Therefore, Guessing Entropy must be used as the definitive measure of attack success.

Second, the results demonstrate that sequence-based memory models like LSTMs actively degrade the location-specific features needed for key recovery, as hardware power leakage is an instantaneous spatial event.

Third, I observed the dual nature of baseline models. The MLP completely failed against the Boolean masking of the ASCAD dataset (0.43% accuracy). However, through data regularization (Standard Scaling and Batch Normalization) and the statistical brute force of 25,000 attack traces, the same baseline MLP successfully overcame the severe hardware clock jitter of the AES_HD dataset to recover the secret key.

Ultimately, I concluded that while countermeasures like masking remain effective against basic networks, raw hardware implementations remain critically vulnerable to both CNNs and highly optimized MLPs.

A Appendix: Implementation Codes

A.1 Code A: Dataset Download and Extraction

```
# Download preprocessed AES_HD from GitHub (small zip file)
!wget https://github.com/gabzai/Methodology-for-efficient-CNN-
    architectures-in-SCA/raw/master/AES_HD/AES_HD_dataset.zip

# Unzip
!unzip AES_HD_dataset.zip

# Check what's inside
!ls -lh
```

Listing 1: Downloading and Unzipping AES_HD Dataset

A.2 Code B: Power Trace Visualization

```
import matplotlib.pyplot as plt
import numpy as np

X_train_raw = np.load('/content/AES_HD_dataset/
    profiling_traces_AES_HD.npy')

plt.figure(figsize=(12, 4))
plt.plot(X_train_raw[0], color='blue', linewidth=0.8, label='Trace 1'
)
plt.plot(X_train_raw[1], color='orange', linewidth=0.8, alpha=0.7,
    label='Trace 2')
plt.axvline(x=965, color='red', linestyle='—', linewidth=1.5, label=
    'Leakage Point (~timepoint 965)')
plt.title('Sample AES_HD Power Traces (Two Encryptions)')
plt.xlabel('Timepoint (samples)')
plt.ylabel('Power Consumption (ADC units)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('sample_traces.png', dpi=150)
plt.show()
```

Listing 2: Visualizing AES_HD Power Traces

A.3 Code C: Multi-Layer Perceptron on ASCAD Dataset

```
# MLP ON ASCAD
import h5py
import numpy as np
from sklearn.preprocessing import StandardScaler

f = h5py.File('/content/ASCAD.h5', 'r')
print(list(f.keys()))
print(list(f['Profiling_traces'].keys()))
print(list(f['Attack_traces'].keys()))

X_train = f['Profiling_traces']['traces'][:].astype('float32')
Y_train = f['Profiling_traces']['labels'][:]
print("Shape of X_train: ", X_train.shape)
print("Shape of Y_train: ", Y_train.shape)

X_test = f['Attack_traces']['traces'][:].astype('float32')
Y_test = f['Attack_traces']['labels'][:]
print("Shape of X_test: ", X_test.shape)
print("Shape of Y_test: ", Y_test.shape)

X_train = X_train[:, 45:80]
X_test = X_test[:, 45:80]

scalar = StandardScaler()
X_train = scalar.fit_transform(X_train)
X_test = scalar.transform(X_test)

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam

Y_train_cat = to_categorical(Y_train, num_classes=256)
Y_test_cat = to_categorical(Y_test, num_classes=256)

mlp_model = Sequential([
    Dense(200, activation='relu', input_shape=(35,)),
    Dropout(0.3),
    Dense(200, activation='relu'),
    Dropout(0.3),
    Dense(200, activation='relu'),
    Dropout(0.3),
    Dense(256, activation='softmax')
])
```



```
mlp_model.compile(optimizer=Adam(learning_rate=0.001),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

h_mlp = mlp_model.fit(X_train, Y_train_cat, epochs=50, batch_size
                      =256,
                      validation_data=(X_test, Y_test_cat), verbose
                      =1)
loss, acc = mlp_model.evaluate(X_test, Y_test_cat, verbose=0)
print(f"\nMLP Test Accuracy: {acc*100:.2f}%")
```

Listing 3: MLP on ASCAD Dataset

A.4 Code D: Multi-Layer Perceptron on AES_HD Dataset

```
# MLP on AES_HD
import numpy as np
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout,
    BatchNormalization, Input
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping,
    ReduceLROnPlateau

X_train = np.load('/content/AES_HD_dataset/profiling_traces_AES_HD.
    npy')
Y_train = np.load('/content/AES_HD_dataset/profiling_labels_AES_HD.
    npy')
X_test = np.load('/content/AES_HD_dataset/attack_traces_AES_HD.npy')
Y_test = np.load('/content/AES_HD_dataset/attack_labels_AES_HD.npy')

Y_train = Y_train.flatten().astype(int)
Y_test = Y_test.flatten().astype(int)
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')

hw = np.vectorize(lambda x: bin(int(x)).count('1'))
Y_train_hw = hw(Y_train)
Y_test_hw = hw(Y_test)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

Y_train_cat = to_categorical(Y_train_hw, num_classes=9)
Y_test_cat = to_categorical(Y_test_hw, num_classes=9)

mlp_model = Sequential([
    Input(shape=(1250,)),
    Dense(256, activation='relu'),
    BatchNormalization(),
    Dropout(0.2),
    Dense(256, activation='relu'),
    BatchNormalization(),
    Dropout(0.2),
    Dense(256, activation='relu'),
    BatchNormalization(),
```

```

        Dropout(0.2),
        Dense(9, activation='softmax')
])

mlp_model.compile(optimizer=Adam(learning_rate=0.001),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

early_stop = EarlyStopping(monitor='val_accuracy', patience=15,
                           restore_best_weights=True, verbose=1)
reduce_lr = ReduceLROnPlateau(monitor='val_accuracy', factor=0.5,
                              patience=5, min_lr=1e-5, verbose=1)

h_mlp = mlp_model.fit(X_train, Y_train_cat, epochs=100, batch_size
                      =256,
                      validation_data=(X_test, Y_test_cat),
                      callbacks=[early_stop, reduce_lr], verbose=1)

loss, acc = mlp_model.evaluate(X_test, Y_test_cat, verbose=0)
print(f"MLP Test Accuracy: {acc*100:.2f}%")

```

Listing 4: MLP on AES_HD Dataset

A.5 Code E: 1D Convolutional Neural Network on AES_HD Dataset

```
# CNN on AES_HD
import numpy as np
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten,
    Dense, Dropout, BatchNormalization, Input
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping,
    ReduceLROnPlateau

X_train = np.load('/content/AES_HD_dataset/profiling_traces_AES_HD.
    npy')
Y_train = np.load('/content/AES_HD_dataset/profiling_labels_AES_HD.
    npy')
X_test = np.load('/content/AES_HD_dataset/attack_traces_AES_HD.npy')
Y_test = np.load('/content/AES_HD_dataset/attack_labels_AES_HD.npy')

Y_train = Y_train.flatten().astype(int)
Y_test = Y_test.flatten().astype(int)
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')

hw = np.vectorize(lambda x: bin(int(x)).count('1'))
Y_train_hw = hw(Y_train)
Y_test_hw = hw(Y_test)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

Y_train_cat = to_categorical(Y_train_hw, num_classes=9)
Y_test_cat = to_categorical(Y_test_hw, num_classes=9)

X_train_cnn = X_train[..., np.newaxis]
X_test_cnn = X_test[..., np.newaxis]

cnn_model = Sequential([
    Input(shape=(1250, 1)),
    Conv1D(filters=8, kernel_size=11, activation='relu', padding='
    same'),
    BatchNormalization(),
    MaxPooling1D(pool_size=4, strides=4),
```

```

        Conv1D(filters=16, kernel_size=11, activation='relu', padding='
        same'),
        BatchNormalization(),
        MaxPooling1D(pool_size=4, strides=4),
        Conv1D(filters=32, kernel_size=11, activation='relu', padding='
        same'),
        BatchNormalization(),
        MaxPooling1D(pool_size=4, strides=4),
        Flatten(),
        Dense(64, activation='relu'),
        BatchNormalization(),
        Dropout(0.4),
        Dense(32, activation='relu'),
        BatchNormalization(),
        Dropout(0.4),
        Dense(9, activation='softmax')
    ])

cnn_model.compile(optimizer=Adam(learning_rate=0.001),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

early_stop = EarlyStopping(monitor='val_accuracy', patience=15,
                           restore_best_weights=True, verbose=1)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                              patience=5, min_lr=1e-5, verbose=1)

h_cnn = cnn_model.fit(X_train_cnn, Y_train_cat, epochs=100,
                      batch_size=256,
                      validation_data=(X_test_cnn, Y_test_cat),
                      callbacks=[early_stop, reduce_lr], verbose=1)

```

Listing 5: 1D-CNN on AES_HD Dataset

A.6 Code F: Hybrid CNN-LSTM on AES_HD Dataset

```
# CNN-LSTM HYBRID ON AES_HD
import numpy as np
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, LSTM, MaxPooling1D, Dense,
    Dropout, BatchNormalization, Input
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping,
    ReduceLROnPlateau

X_train = np.load('/content/AES_HD_dataset/profiling_traces_AES_HD.
    npy')
Y_train = np.load('/content/AES_HD_dataset/profiling_labels_AES_HD.
    npy')
X_test = np.load('/content/AES_HD_dataset/attack_traces_AES_HD.npy')
Y_test = np.load('/content/AES_HD_dataset/attack_labels_AES_HD.npy')

Y_train = Y_train.flatten().astype(int)
Y_test = Y_test.flatten().astype(int)
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')

hw = np.vectorize(lambda x: bin(int(x)).count('1'))
Y_train_hw = hw(Y_train)
Y_test_hw = hw(Y_test)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

Y_train_cat = to_categorical(Y_train_hw, num_classes=9)
Y_test_cat = to_categorical(Y_test_hw, num_classes=9)

X_train_reshaped = X_train[..., np.newaxis]
X_test_reshaped = X_test[..., np.newaxis]

hybrid_model = Sequential([
    Input(shape=(1250, 1)),
    Conv1D(filters=16, kernel_size=11, activation='relu', padding='
    same'),
    BatchNormalization(),
    MaxPooling1D(pool_size=4, strides=4),
```

```

        Conv1D(filters=32, kernel_size=11, activation='relu', padding='
        same'),
        BatchNormalization(),
        MaxPooling1D(pool_size=4, strides=4),
        LSTM(64, return_sequences=False),
        BatchNormalization(),
        Dropout(0.4),
        Dense(64, activation='relu'),
        BatchNormalization(),
        Dropout(0.4),
        Dense(9, activation='softmax')
    ])

    hybrid_model.compile(optimizer=Adam(learning_rate=0.001),
                        loss='categorical_crossentropy',
                        metrics=['accuracy'])

    early_stop = EarlyStopping(monitor='val_accuracy', patience=10,
                                restore_best_weights=True, verbose=1)
    reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                                   patience=4, min_lr=1e-5, verbose=1)

    h_hybrid = hybrid_model.fit(X_train_reshaped, Y_train_cat, epochs
                                =100, batch_size=256,
                                validation_data=(X_test_reshaped,
                                Y_test_cat),
                                callbacks=[early_stop, reduce_lr],
                                verbose=1)

```

Listing 6: Hybrid CNN-LSTM on AES_HD Dataset

References

- [1] P. Kocher, J. Jaffe, and B. Jun, “Differential Power Analysis,” in *Advances in Cryptology — CRYPTO ’99*, Berlin, Heidelberg: Springer, 1999, pp. 388–397.
- [2] H. Maghrebi, T. Porteboeuf, and J.-L. Danger, “Breaking Cryptographic Implementations Using Deep Learning Techniques,” in *Security, Privacy, and Applied Cryptography Engineering (SPACE)*, Cham: Springer, 2016, pp. 3–26.
- [3] R. Benadjila, E. Prouff, R. Strullu, E. Cagli, and C. Dumas, “Study of Deep Learning Techniques for Side-Channel Analysis and Introduction to ASCAD Database,” *Cryptology ePrint Archive*, Report 2018/053, 2018. [Online]. Available: https://www.data.gouv.fr/s/resources/ascad/20180530-163000/ASCAD_data.zip
- [4] G. Zaid, L. Bossuet, A. Habrard, and A. Venelli, “Methodology for Efficient

CNN Architectures in Profiling Attacks,” *IACR Transactions on Cryptographic Hardware and Embedded Systems (CHES)*, vol. 2020, no. 1, pp. 1–36, Nov. 2019.

[Online]. Available: <https://github.com/gabzai/Methodology-for-efficient-CNN-archi>
[raw/master/AES_HD/AES_HD_dataset.zip](https://github.com/gabzai/Methodology-for-efficient-CNN-archi/raw/master/AES_HD/AES_HD_dataset.zip)

- [5] R. Benadjila, E. Prouff, R. Strullu, E. Cagli, and C. Dumas, “Deep Learning for Side-Channel Analysis and Introduction to ASCAD Database,” *Journal of Cryptographic Engineering*, vol. 10, pp. 163–188, 2020.